California State Polytechnic University, Pomona

Cal Poly Pomona Autonomous Vehicle Lab

IGVC 2026 Design Report

PLACEHOLDER_ROBOT_NAME

[Insert robot photo: figures/robot_photo.jpg]

I hereby certify that the development of the vehicle, PLACEHOLDER_ROBOT_NAME, as described in this report,
is equivalent to the work involved in a senior design course.

Submitted **[Month Day]**, 2026

Software	Mechanical	Electrical
[Parsa Ghasemi (Lead)]	**[Name (Lead)]**	**[Name (Lead)]**
[Caleb Hylkema]	**[Name]**	**[Name]**
[Name]	**[Name]**	**[Name]**
[Name]	**[Name]**	**[Name]**

Faculty Advisor: **[Dr. Advisor Name, Department of Electrical and Computer Engineering]**

Faculty Signature: _____ Date: _____

Contents

1. Design Process, Team Identification, and Organization

1.1 Introduction

The Cal Poly Pomona Autonomous Vehicle Lab (AVL) at California State Polytechnic University, Pomona presents PLACEHOLDER_ROBOT_NAME, our entry for the 2026 Intelligent Ground Vehicle Competition (IGVC) in both the AutoNav and Self-Drive challenges. AVL is an undergraduate research lab within the College of Engineering focused on autonomy, embedded systems, and ground-vehicle perception. The IGVC subteam consists of **[N]** students organized across three disciplines: software, mechanical, and electrical, advised by **[Dr. Advisor Name]**.

PLACEHOLDER_ROBOT_NAME is a fully autonomous differential-drive tracked vehicle built on the AVL's drive-by-wire research platform (AndyMark Raptor chassis). The robot fuses a Velodyne VLP-16 3D LiDAR, three Stereolabs ZED X stereo cameras (front/left/right), and an Xsens MTi-680G IMU+GNSS for perception and localization, with all autonomy software running on an NVIDIA Jetson Orin under ROS 2 Humble.

1.2 Team Organization

The IGVC team is led by a project captain and divided into three subteams reporting to the captain. The software subteam owns perception, localization, planning, control, and firmware. The mechanical subteam owns chassis, drivetrain, sensor mounting, and weatherproofing. The electrical subteam owns power distribution, wiring, and the safety interlock circuit. Weekly all-hands meetings keep subteams synchronized; biweekly integration days exercise the full vehicle on the test course.

1.3 Design Process

The team follows an iterative cycle of *research* → *design* → *prototype* → *test*, executed in four phases over the academic year:

1. **Requirements and rules analysis.** Re-read of the IGVC 2026 rulebook, review of recent winning design reports (notably Sooner Robotics 2025 and Ville Robotics 2024), and definition of system-level requirements for chassis, sensor suite, compute, and software stack.
2. **Subsystem development.** Each subteam develops its components independently — mechanical CAD, electrical schematics, ROS 2 packages, and Teensy firmware.
3. **Integration.** Subsystems are integrated incrementally: firmware on the bench (Phases 1–7 of motor characterization), then attached to the actuator node, then sensor fusion, then full Nav2 stack.
4. **Field testing.** Iterative outdoor testing on the **[CPP test area]**, with data-driven tuning of HSV perception thresholds, EKF covariances, costmap parameters, and MPPI critic weights.

2. System Architecture

PLACEHOLDER_ROBOT_NAME is organized into five functional layers: **sensing**, **perception**, **localization**, **navigation**, and **actuation**. All software runs on the Jetson Orin under ROS 2 Humble with CycloneDDS middleware. A Teensy 4.1 microcontroller bridges the Jetson (USB serial) to two REV SparkMAX motor controllers (CAN bus at 1 Mbit/s). Figure 1 shows the high-level dataflow.

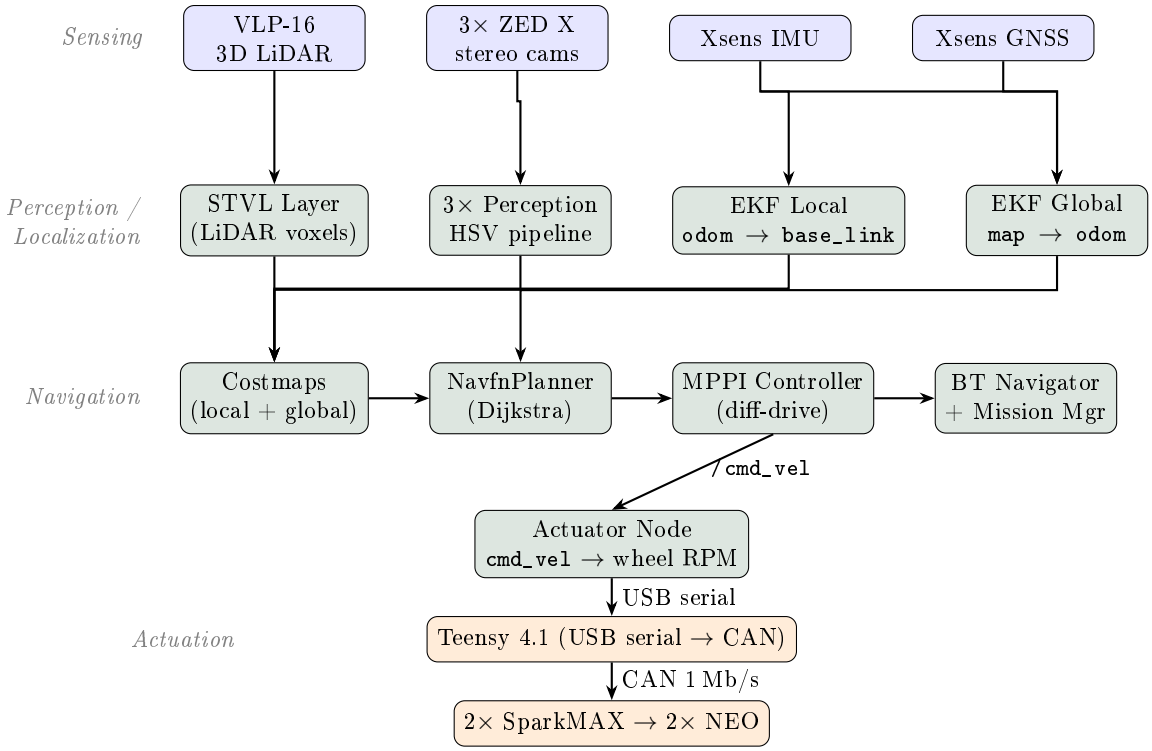

Figure 1: PLACEHOLDER_ROBOT_NAME system architecture. Sensors feed perception and localization; Nav2 composes costmaps, plans (NavfnPlanner), and follows the plan (MPPI Controller). The actuator node converts `/cmd_vel` to per-wheel RPM and forwards it to the Teensy, which writes velocity setpoints to two SparkMAX motor controllers over CAN.

2.1 Significant Components and Bill of Materials

Table 1: Bill of Significant Components (estimated team cost)

Qty	Component	Subsystem	Cost
1	NVIDIA Jetson Orin (JetPack 6, ROS 2 Humble)	Compute	**[$2,000]**
2	REV NEO Brushless Motor (REV-21-1650)	Drivetrain	**[$120]**
2	REV SparkMAX Motor Controller (FW 26.1.4)	Drivetrain	**[$180]**
1	Teensy 4.1 + SN65HVD230 CAN transceiver	Firmware bridge	**[$35]**
1	Velodyne VLP-16 3D LiDAR	Perception	**[$4,000]**
3	Stereolabs ZED X stereo cameras	Perception	**[$4,500]**
1	ZED Link Quad GMSL2 capture card	Perception	**[$300]**
1	Xsens MTi-680G IMU + GNSS	Localization	**[$2,500]**
1	AndyMark Raptor tracked chassis	Mechanical	**[$1,800]**
1	Battery + power distribution	Power	**[$500]**
1	E-Stop system, safety beacon, wiring, misc.	Safety / Misc.	**[$400]**
		Total estimated cost	**[$16,335]**

3. Effective Innovations

3.1 MPPI Predictive Controller for Diff-Drive Outdoor Navigation

PLACEHOLDER_ROBOT_NAME uses Nav2's MPPI (Model Predictive Path Integral) controller in place of the more common Regulated Pure Pursuit. MPPI samples 2,000 rollout trajectories per 20 Hz control cycle (56 timesteps, 50 ms model Δt), scores them against eight plugin-based critics (constraint, cost, goal, goal-angle, path-align, path-follow, path-angle, prefer-forward), and emits the noise-weighted optimal command. Unlike pursuit controllers that only *follow* a path, MPPI *generates* trajectories that deviate from the global plan to avoid newly observed obstacles — critical for IGVC's randomly-placed barrels and dynamic pedestrians in the Self-Drive course. The tuning parameters (critic weights, temperature, gamma) were adopted from Clearpath's production A300/J100/W200 outdoor diff-drive configurations and refined against measured firmware limits (Section 6.5).

3.2 Three-Camera Semantic Costmap with STVL Fusion

The local costmap composes three independent sources: a Velodyne VLP-16 stream through the Spatio-Temporal Voxel Layer (STVL), and three semantic streams (front/left/right ZED X) through the `kiwicampus/semantic_segmentation_layer`. STVL replaces the standard `VoxelLayer` because the VLP-16's 16-beam ±15° vertical FOV leaves angular gaps that defeat ray-trace clearing when the robot is stationary; STVL clears via time-based voxel decay (5 s) and a frustum-accelerated decay term that handles the sparse-scan case cleanly. The three semantic streams provide near-360° camera coverage of lane paint and barrels, which the LiDAR alone cannot classify by color.

3.3 Dual-EKF Localization with Differential GPS Fusion

Localization uses the canonical `robot_localization` dual-EKF pattern, but with **GPS fused in differential mode** into the global EKF. The global filter integrates per-step GPS deltas rather than absolute fixes, which prevents costmap jumps from the ±2–5 m noise typical of consumer-grade GNSS. A 6σ Mahalanobis outlier-rejection threshold protects the filter from multipath spikes. The ZED X visual-inertial odometry is fused into the local EKF for **yaw only**, not position — the ZED wrapper hard-codes its child frame to `<camera>_camera_link` and `robot_localization`'s differential pose path applies only a frame rotation (no lever-arm correction), so fusing the ZED's x/y would inject a fake lateral velocity of $\omega \times r$ on every turn (≈0.34 m/s at 0.5 rad/s with our 0.68 m forward mount).

4. Mechanical Design

[MECHANICAL DESIGN SECTION (target: 2–3 pages).

To include the mechanical subteam's PDF report, place it at `figures/mechanical_report.pdf` and uncomment the `\includepdf` line below. The included PDF should cover: chassis frame and housing design, the AndyMark Raptor tracked drivetrain (12.75:1 gearbox, 20T drive pulleys, 0.7366 m track gauge), wheel/tire selection, suspension (if any), weatherproofing, sensor mast and mounting hardware, payload mounting, CAD renderings, and overall vehicle dimensions and mass.]

For the integrated report, the mechanical section should follow the structure used by recent winning teams (Sooner Robotics 2025, Ville Robotics 2024):

- **4.1 Overview** — design goals (electrical-bay access, weatherproofing, sensor sightlines)
- **4.2 Frame and Chassis** — AndyMark Raptor base, materials, dimensions
- **4.3 Drivetrain** — NEO motors, gearbox ratio, ground-speed calculation
- **4.4 Sensor Mast and Mounting** — ZED ×3, VLP-16, GNSS antenna placement

- **4.5 Weatherproofing** — panels, gaskets, enclosure IP rating

5. Electrical and Power Design

[ELECTRICAL DESIGN SECTION (target: 2–3 pages).

To include the electrical subteam's PDF report, place it at `figures/electrical_report.pdf` and uncomment the `\includepdf` line below. The included PDF should cover: power distribution diagram (separate 19 V Jetson rail and 12 V/CAN rail so motor inrush cannot brown out the compute), battery selection and runtime analysis, voltage rails and DC-DC converters, fusing, the three-input E-stop interlock circuit (physical button, wireless relay, software interlock), safety light driver, and the electrical BOM.]

Firmware. The Teensy 4.1 translates ASCII RPM commands into SparkMAX CAN velocity setpoints at 1 Mbit/s and echoes encoder feedback at 50 Hz. Three independent timeouts protect the drivetrain: a 300 ms serial watchdog on the Teensy, a 100 ms CAN heartbeat timeout per SparkMAX, and a firmware clamp of ±4600 RPM (1.53 m/s ground speed, below the 5 mph IGVC cap).

Recommended subsections (following Sooner Robotics 2025):

- **5.1 Overview** — electronic suite block diagram
- **5.2 Power Distribution** — battery, voltage rails, runtime analysis
- **5.3 Motor Control** — SparkMAX configuration on the CAN bus
- **5.4 Safety Light** — wiring, solid/flashing logic
- **5.5 Emergency Stop** — physical button + wireless E-stop + software interlock; circuit diagram showing how all three gate motor power

The integrated software-side view of E-stop and safety enforcement is described in Section 8.

6. Software Systems

6.1 Overview

The full autonomy stack runs on the Jetson Orin under ROS 2 Humble with CycloneDDS middleware. The software repository (`Paarseus/IGVC_ROS2`, "AVROS") is organized into seven custom packages plus upstream Nav2, `robot_localization`, the `kiwicampus/semantic_segmentation_layer` costmap plugin, and `spatio_temporal_voxel_layer`.

Table 2: AVROS ROS 2 Packages

Package	Purpose
`avros_msgs`	ActuatorCommand / ActuatorState messages, PlanRoute service
`avros_bringup`	Launch files, URDF, YAML configs, Nav2 params, BT XMLs, RViz config
`avros_control`	`actuator_node`: `cmd_vel` → diff-drive inverse → Teensy USB serial
`avros_perception`	`perception_node`: ZED RGB+cloud → pluggable pipeline → semantic mask
`avros_navigation`	`mission_manager`: waypoint cursor, lat/lon → map frame, fires NavigateToPose
`avros_webui`	FastAPI/WebSocket phone joystick → ActuatorCommand
`avros_sim`	Webots simulation: `cpp_campus.wbt` world, vehicle driver plugin

6.2 Perception: Lane and Obstacle Detection

LiDAR (Velodyne VLP-16). The VLP-16 publishes a 360° point cloud at 10 Hz into Nav2's `spatio_temporal_voxel_layer` (STVL), which maintains a 0.1 m voxel grid with 5 s linear voxel decay. A minimum obstacle height of 0.2 m filters ground hits and low grass while keeping short

barrels and pedestrians. Detection range is 1–15 m local / 1–50 m global.

Vision (3× ZED X). Each ZED X publishes rectified HD1080 color and a COMPACT-resolution organized point cloud at 10 Hz. The `avros_perception` package runs one `perception_node` per camera; each subscribes to its camera's RGB and cloud topics through an `ApproximateTimeSynchronizer` (20 ms slop) and runs a pluggable pipeline. Pipelines are selected at launch:

- `hsv` (production default) — three-class HSV thresholding (white lane, orange barrel, pothole) with an adaptive value-channel floor ($V_{\text{floor}} = \mu_V + k\sigma_V$, recomputed every N frames) and a polygonal sky-rejection ROI. Class IDs: 1 = lane, 2 = barrel, 3 = pothole.
- `sooner25` — single-class inverted threshold (S, V upper bounds, then `255-mask`) so any saturated or bright pixel becomes obstacle. Better for white-on-asphalt; `hsv` is preferred for white-on-grass.
- `stub` — injects a debug stripe to exercise the costmap-to-planner contract without real images.

Every pipeline outputs the four-topic contract consumed by the `kiwicampus/semantic_segmentation_layer`: a mono8 class-ID mask, a mono8 confidence plane, the organized point cloud (relayed), and a latched `vision_msgs/LabelInfo`. All thresholds are runtime-tunable via `ros2 param set` for field calibration.

6.3 Localization and Sensor Fusion

Localization uses two instances of `robot_localization`'s EKF, the standard pattern for GPS-enabled outdoor robots (Figure 2). Both filters run at 30 Hz in 2D mode.

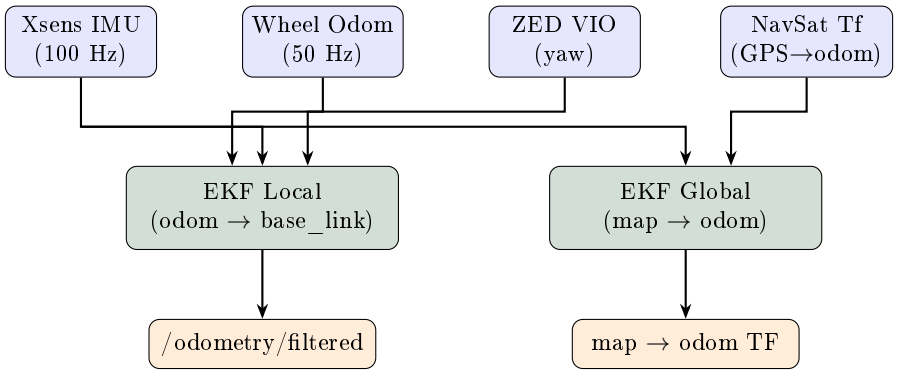

Figure 2: Dual-EKF sensor fusion. The local filter produces smooth odometry for the controller; the global filter anchors the local frame to GPS coordinates. ZED VIO is fused for yaw only to avoid lever-arm bias; GPS is fused in differential mode so noisy consumer-grade fixes integrate as deltas rather than creating costmap jumps.

The **local EKF** (`odom` → `base_link`) fuses three sources: the Xsens IMU (roll, pitch, yaw, and angular velocities), wheel odometry from the actuator node (linear velocity v_x and yaw rate $\dot{\psi}$ from diff-drive forward kinematics, with v_y marked unobservable), and the ZED X VIO yaw (differential, avoiding the lever-arm bias). The **global EKF** (`map` → `odom`) fuses the IMU stream with GPS odometry from the `navsat_transform_node`, integrated in differential mode with a 6σ outlier-rejection threshold.

6.4 Path Planning and Navigation

The Nav2 stack provides path planning, trajectory generation, and behavior orchestration. Key plugins and parameters for the Humble production build are listed in Table 3.

Table 3: Nav2 Configuration (`nav2_params_humble.yaml`)

Component	Configuration
Planner	`nav2_navfn_planner::NavfnPlanner` (Dijkstra) — chosen over SmacPlannerHybrid because the IGVC lane spacing (2–3 m) is narrower than the Hybrid planner's 2.31 m minimum turning radius
Controller	`nav2_mppi_controller::MPPIController` — DiffDrive motion model, 2000 rollouts, 56 timesteps, model Δt 50 ms, 8 active critics
Local costmap	50×50 m rolling, 0.2 m resolution; `stvl_layer` + `semantic_layer` (front/left/right) + `inflation_layer`
Global costmap	100×100 m rolling, 0.2 m resolution; `obstacle_layer` (VLP-16) + `inflation_layer`
Footprint	Rectangular 1.00×0.74 m, $r_{\text{inscr}} = 0.5$ m
Inflation	$r_{\text{infl}} = 0.65$ m, $k_{\text{scale}} = 3.0$ (smooth gradient for MPPI critic)
Goal tolerance	xy 2.0 m (IGVC waypoint acceptance), yaw 0.5 rad
Controller frequency	20 Hz; planner replans at 1 Hz via BT `RateController`

Waypoint orchestration. The `mission_manager` node in `avros_navigation` loads the IGVC competition waypoints from `waypoints.yaml` as a lat/lon list, converts each to the map frame via the `robot_localization /fromLL` service, and fires `NavigateToPose` action goals to Nav2 sequentially. The cursor advances when the robot is within a 2.0 m acceptance radius (IGVC rule), and `ABORTED`/`CANCELED` failures fall through to the next waypoint rather than halting the mission.

Behavior tree. The active BT (`navigate_igvc_autonav_humble.xml`) wraps the standard plan-follow pipeline with several IGVC-specific guards. An outer `Timeout` of 45 s margins the 60 s hold-up-traffic disqualification rule. A `RecoveryNode` retries the pipeline up to three times. Recovery actions form a `RoundRobin` that escalates each attempt: clear costmap around robot → wait 2 s for stale semantic cells to decay → short backup (1.5 m at 0.08 m/s) → crawl forward (0.15 m at 0.10 m/s). Spin-in-place is intentionally omitted: the 1.00×0.74 m rectangular footprint sweeps a 2.48 m diameter circle when off-center, exceeding the half-lane width and risking the IGVC 3-inch white-tape crossing penalty.

6.5 Drive-by-Wire Control

The `actuator_node` (Figure 1, lower right) is the bridge from Nav2 to the physical drivetrain. It subscribes to `/cmd_vel` (from Nav2, teleop, or the velocity smoother) and `/avros/actuator_command` (from the webui for manual override). Each control loop tick at 50 Hz:

1. **Source arbitration.** A fresh `ActuatorCommand` takes priority over `cmd_vel`; the source falls back to a hard zero if neither has been updated within `cmd_timeout_s` (0.5 s).
2. **Slew-rate limit.** Linear: 0.5 m/s^2 accel / 1.5 m/s^2 brake. Angular: 1.0 rad/s^2. Protects the 12 V rail from motor inrush (which previously caused Jetson brown-outs) and gives smooth ride feel.
3. **IMU-based assist.** When $|\omega_{\text{cmd}}| < 0.05$ rad/s, the current IMU yaw is locked and a P-controller ($K_p = 1.5$) drives the robot straight regardless of surface friction asymmetry — "heading hold." When $|\omega_{\text{cmd}}| \geq 0.05$ rad/s, the IMU yaw rate becomes feedback ($K_p = 0.3$) so a 0.5 rad/s command produces 0.5 rad/s on any surface — "gyro-stabilized turn."
4. **Diff-drive inverse.** $(v, \omega) \rightarrow$ (L_rpm, R_rpm) using `track_width_m = 0.7366 m`, `m_per_motor_rev = 0.01994 m`.
5. **Serial write.** ASCII command `L<rpm> R<rpm>` is written over USB serial at 115200 baud to the Teensy 4.1.

In parallel, the actuator node integrates wheel encoder feedback into a 50 Hz `/wheel_odom`

`nav_msgs/Odometry` message that the local EKF consumes.

6.6 Simulation

Development and regression testing use the `avros_sim` package, which runs Webots with a `cpp_campus.wbt` world imported from the Cal Poly Pomona OpenStreetMap extract. A custom Webots driver publishes simulated VLP-16, ZED, and GNSS topics that the rest of the stack consumes unchanged, enabling perception and Nav2 development without field time.

7. Cyber Security Analysis

PLACEHOLDER_ROBOT_NAME follows the NIST Risk Management Framework (RMF) approach: identify threats and impacted subsystems, classify by confidentiality / integrity / availability impact, and apply NIST 800-53 controls.

Table 4: Threat Model and Mitigations

Subsystem / Threat	Mitigation (NIST control)	C/I/A
Onboard network breach	Dedicated 192.168.13.0/24 subnet, WPA2 (SC-7)	L/H/H
ROS 2 topic injection	CycloneDDS with DDS Security; network isolation (AC-3)	L/H/H
WebSocket joystick hijack	HTTPS/WSS, fail-safe disconnect → E-stop (AC-3)	L/H/H
GPS spoofing / multipath	Differential GPS fusion + 6σ Mahalanobis rejection (SI-4, SI-7)	L/M/M
SSH access to Jetson	Key-only authentication; password login disabled (AC-3, IA-2)	H/H/M
Git repository tampering	Branch protection, PR review (CM-5)	M/H/L

Defense in depth. Safety-critical motion is gated by three independent stop mechanisms: the physical E-stop button on the vehicle rear, the wireless E-stop relay held by the judges during competition runs, and a software interlock in the actuator node that zeroes `cmd_vel` on stale input. Any one of the three removes power from the motors; all three default to engaged on communication loss. Manual `ActuatorCommand` input from the webui always takes priority over autonomous `cmd_vel`, so a human operator can override Nav2 at any time.

8. Vehicle Analysis

8.1 Safety, Reliability, and Durability

Safety is enforced at every layer: a hardware speed cap (4600 RPM → 1.53 m/s) in the Teensy firmware, a 300 ms serial watchdog on the Teensy, a 100 ms CAN heartbeat timeout per SparkMAX, a software slew-rate limit in the actuator node, and Nav2's MPPI collision critic plus a costmap inflation radius of 0.65 m (inscribed radius plus margin). The MPPI `CostCritic` uses `consider_footprint: true` so the rectangular 1.00×0.74 m footprint is swept along every rollout trajectory rather than only the center point — catching trajectories that would clip a thin painted lane line at high steering angles.

8.2 Failure Points and Resolutions

Table 5: Known Failure Modes and Mitigations

Layer	Failure	Mitigation
Mechanical	Fasteners loosen under vibration	Loctite + Nyloc nuts; periodic torque checks
Electrical	Motor inrush brown-out on shared rail	Separate Jetson power rail
Electrical	Wireless E-stop link loss	Heartbeat timeout cuts motor relay power
Sensing	Encoder dropout	Local EKF reverts to IMU prediction
Sensing	GPS multipath / dropout	6σ outlier rejection; differential fusion
Perception	HSV threshold drift across lighting	Adaptive V-floor; live-tunable HSV bounds
Costmap	Stale semantic cells linger	STVL 5 s decay; semantic raytrace clearing
Software	Stale `cmd_vel`	300 ms watchdog forces motor stop

8.3 Testing Methodology and Version Control

Software is developed in a Git repository (`github.com/Paarseus/IGVC_ROS2`) with feature branches, pull-request review, and automated linting via `colcon test`. All parameters — perception thresholds, EKF covariances, Nav2 gains, MPPI critic weights, actuator slew limits — live in version-controlled YAML, so the system is reproducible from a clean checkout.

Testing progresses through four stages: bench (subsystem in isolation), simulation (Webots full stack), low-speed field tests on a flat surface, and full-speed field tests on the IGVC practice course.

9. Unique AutoNav and Self-Drive Software

PLACEHOLDER_ROBOT_NAME competes in both AutoNav and Self-Drive. The two challenges share most of the software stack: HSV lane detection, LiDAR obstacle avoidance, dual-EKF localization, MPPI control, and the actuator node are identical. The differences are in the orchestration layer.

9.1 AutoNav-Specific Features

- **Lane following:** HSV white-lane mask from each of three ZED X cameras feeds the local costmap's `semantic_layer` as lethal cells; MPPI generates rollouts that stay inside the lane corridor.
- **Obstacle avoidance:** VLP-16 STVL marks barrels and signs in both costmaps; MPPI's `PreferForwardCritic` and `PathFollowCritic` bias rollouts toward forward motion along the global plan when no obstacles intrude.
- **Waypoint navigation:** `mission_manager` converts lat/lon waypoints from `waypoints.yaml` into map-frame goals via the `/fromLL` service and fires sequential `NavigateToPose` actions with a 2 m acceptance radius (IGVC rule).
- **Ramp handling:** The 2D-mode EKF assumption holds for the IGVC ramp incline; pitch is monitored from the IMU. MPPI's cost-regulated velocity scaling automatically slows the robot near costmap inflation.

9.2 Self-Drive-Specific Features

- **Teleoperation:** `avros_webui` provides a phone-based proportional joystick via WebSocket; the same node is used for bench testing and Self-Drive demonstration runs.
- **E-Stop compliance:** Physical, wireless, and software E-stops are independently armed; the wireless relay is verified effective at >100 ft per IGVC qualification.
- **Speed governance:** Hardware-enforced 1.53 m/s ground-speed cap (3.4 mph, well under the 5 mph IGVC limit); software slew-rate limit ensures smooth starts and stops.

- **Heading stability:** The actuator node's IMU-based heading hold gives straight-line driving on uneven surfaces without operator correction.

10. Performance Assessment

Table 6 summarizes the current readiness of each IGVC function as of report submission. Detailed bench and field data are maintained in the `docs/` folder of the repository.

Table 6: IGVC Function Status

Category	Function	Status
Qualification	Dimensions, E-stops, safety light	**[Complete]**
	Hardware-governed maximum speed	Complete
	Lane following demonstration	**[Testing]**
	Obstacle avoidance demonstration	**[Testing]**
AutoNav	Autonomous control	**[Testing]**
	Waypoint navigation	**[Testing]**
	Ramp traversal	**[Not yet tested]**
Self-Drive	Teleoperation	Complete
	Parking maneuvers	**[Not yet tested]**
	Lane changing	**[Not yet tested]**

Table 7: Measured Performance Targets

Parameter	Requirement	Measured
Maximum ground speed (hardware-governed)	≤5 mph	3.4 mph (1.53 m/s)
Wireless E-stop range	≥100 ft	**[measured ft]**
Obstacle detection range (VLP-16, local costmap)	≥2 m	15 m
Obstacle detection range (VLP-16, global costmap)	≥2 m	50 m
Battery runtime	≥5 min	**[measured min]**
Course completion (Webots sim)	≤6 min	**[measured min]**

[Add a short paragraph summarizing current readiness, strengths, and the remaining work plan in the final weeks before competition.]

References

1. S. Macenski, F. Martín, R. White, J. Ginés Clavero, "The Marathon 2: A Navigation System," *IEEE/RSJ IROS*, 2020. (Nav2)
2. S. Macenski et al., "On Use of Nav2 MPPI Controller," ROSCon, 2023.
3. T. Moore and D. Stouch, "A Generalized Extended Kalman Filter Implementation for the Robot Operating System," *Advances in Intelligent Systems and Computing*, 2016. (`robot_localization`)
4. S. Macenski, "`spatio_temporal_voxel_layer`," GitHub, 2019–present.
5. Kiwi Campus, "`semantic_segmentation_layer` Nav2 plugin," GitHub, 2023–present.
6. Sooner Competitive Robotics, "IGVC 2025 Design Report: Twistopher," University of Oklahoma, 2025. http://www.igvc.org/design/2025/7.pdf
7. Ville Robotics, "IGVC 2024 Design Report: ALiEN 5.0," Millersville University, 2024. http://www.igvc.org/design/2024/8.pdf
8. REV Robotics, "SPARK MAX Documentation," 2024. https://docs.revrobotics.com/brushless/spark-max/overview

9. PJRC, "Teensy 4.1 Development Board." https://www.pjrc.com/store/teensy41.html
10. Stereolabs, "ZED X and ZED SDK Documentation." https://www.stereolabs.com/docs
11. Xsens / Movella, "MTi-680G Product Documentation." https://www.movella.com/products/sensor-modules/xsens-mti-680g
12. Velodyne Lidar, "VLP-16 User Manual," 2023.
13. IGVC Rules Committee, "33rd Annual IGVC Official Competition Details, Rules, and Format," 2026. http://www.igvc.org/